

Assignment 2 — Experiments and Investigation

The Socket Exchange — TCP Socket Programming

Peduru Kirthi Kiron 2020CS10366

Pragya Malaiya 2020CS10367

Implementation Note

Before running the experiments, a bug was found and fixed in the server's accept loop: on FreeBSD, a pending connection that is aborted before `accept()` retrieves it raises `ConnectionAbortedError` (`ECONNABORTED`). This crashed the entire server (violating the requirement that one client's failure must not affect others). The fix wraps the `accept()` call in its own `try/except` so the accept loop continues on this error instead of propagating it:

```
while True:
    try:
        sock, addr = server_sock.accept()
    except (ConnectionAbortedError, OSError) as e:
        print(f"accept() failed, continuing: {e}", file=sys.stderr)
        continue
    client_thread = threading.Thread(target=handle_client, args=(sock, addr), daemon=True)
    client_thread.start()
```

Also, `server/run-server`, `client/run-trader`, `client/run-market-data`, and the `src/*.py` files were found to have Windows (CRLF) line endings, which broke the shebang line on FreeBSD. Fixed with:

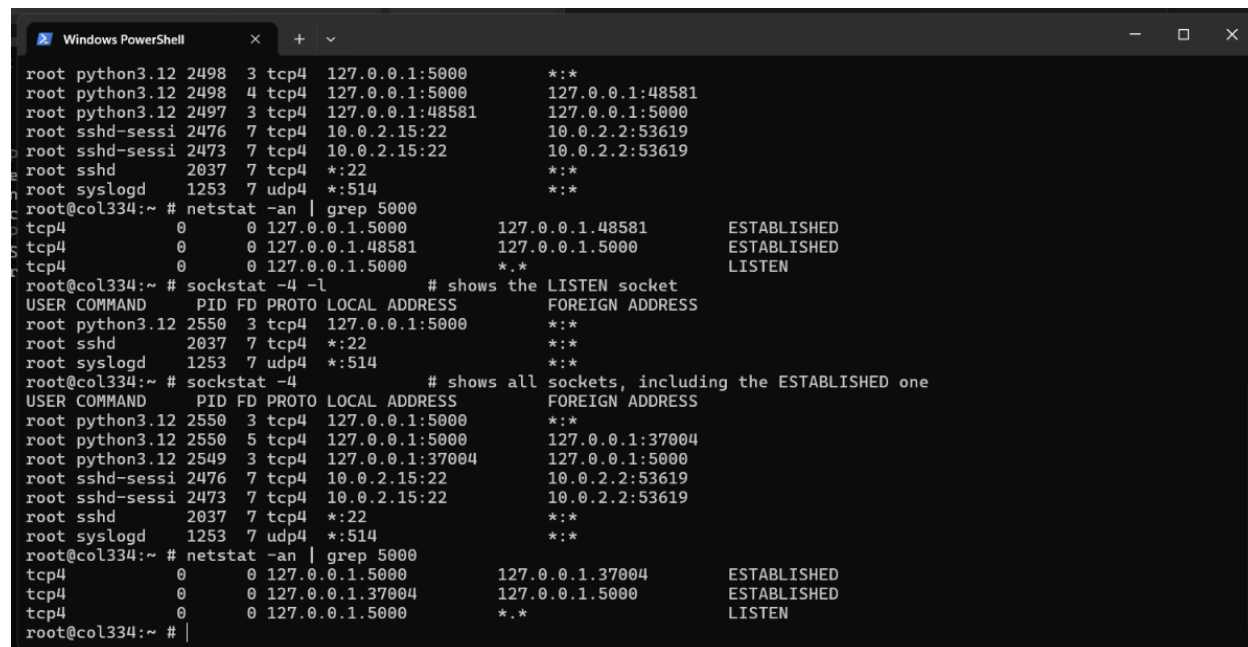
```
sed -i '' 's/\r$//' server/run-server client/run-trader client/run-market-data src/*.py
```

Concurrency model: the server uses one dedicated thread per client connection, spawned from a single `accept()` loop. A global lock protects the shared order book during matching, but is released before any `send()` calls, so a slow or blocked client cannot stall matching for other clients.

Experiment 1 — Listening and Connected Sockets

Question: How does the listening socket differ from the socket representing the connection to the client?

Approach: With the server running and a client connected, `sockstat -4 -l` was used to show only listening sockets, and `sockstat -4` (no `-l`) to show all sockets, including established ones.



```
root python3.12 2498 3 tcp4 127.0.0.1:5000 *: *
root python3.12 2498 4 tcp4 127.0.0.1:5000 127.0.0.1:48581
root python3.12 2497 3 tcp4 127.0.0.1:48581 127.0.0.1:5000
root sshd-sessi 2476 7 tcp4 10.0.2.15:22 10.0.2.2:53619
root sshd-sessi 2473 7 tcp4 10.0.2.15:22 10.0.2.2:53619
root sshd 2037 7 tcp4 *:22 *: *
root syslogd 1253 7 udp4 *:514 *: *
root@col334:~ # netstat -an | grep 5000
tcp4 0 0 127.0.0.1.5000 127.0.0.1.48581 ESTABLISHED
tcp4 0 0 127.0.0.1.48581 127.0.0.1.5000 ESTABLISHED
tcp4 0 0 127.0.0.1.5000 *: * LISTEN
root@col334:~ # sockstat -4 -l # shows the LISTEN socket
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
root python3.12 2550 3 tcp4 127.0.0.1:5000 *: *
root sshd 2037 7 tcp4 *:22 *: *
root syslogd 1253 7 udp4 *:514 *: *
root@col334:~ # sockstat -4 # shows all sockets, including the ESTABLISHED one
USER COMMAND PID FD PROTO LOCAL ADDRESS FOREIGN ADDRESS
root python3.12 2550 3 tcp4 127.0.0.1:5000 *: *
root python3.12 2550 5 tcp4 127.0.0.1:5000 127.0.0.1:37004
root python3.12 2549 3 tcp4 127.0.0.1:37004 127.0.0.1:5000
root sshd-sessi 2476 7 tcp4 10.0.2.15:22 10.0.2.2:53619
root sshd-sessi 2473 7 tcp4 10.0.2.15:22 10.0.2.2:53619
root sshd 2037 7 tcp4 *:22 *: *
root syslogd 1253 7 udp4 *:514 *: *
root@col334:~ # netstat -an | grep 5000
tcp4 0 0 127.0.0.1.5000 127.0.0.1.37004 ESTABLISHED
tcp4 0 0 127.0.0.1.37004 127.0.0.1.5000 ESTABLISHED
tcp4 0 0 127.0.0.1.5000 *: * LISTEN
root@col334:~ #
```

Figure 1: Experiment 1 — `sockstat` output showing the LISTEN socket and the ESTABLISHED socket

Answer: The listening socket (127.0.0.1:5000, foreign address `*:*`) is bound to a local address/port but is not connected to any specific peer — it exists purely to accept new connection attempts and never changes for the lifetime of the server. The moment a client connects, `accept()` returns a **new**, separate file descriptor for that connection: same local address and port, but now paired with the client’s specific address and port (e.g. 127.0.0.1:5000 <-> 127.0.0.1:37004). The original listening socket is untouched and continues to exist alongside it, ready to accept further clients. This is directly visible in the `sockstat` output: the server process (python3.12, PID 2550) owns two distinct sockets — fd 3 (LISTEN, wildcard foreign address) and fd 5 (ESTABLISHED, a specific 4-tuple) — while a third socket (fd 3 on the client’s own process) is the client’s end of that same connection.

Experiment 2 — Observing TCP Connection States

Question: How does the TCP state of the client-server connection change during its lifetime, and what events cause the observed state changes?

Approach: `tcpdump -i lo0 -tttt -n port 5000` was run to capture the raw packet exchange for a single client connection through its full lifecycle (connect → idle → close), giving direct evidence of the events driving each state transition (more precise than periodically polling `netstat`, since on loopback the whole teardown can complete in well under a millisecond).

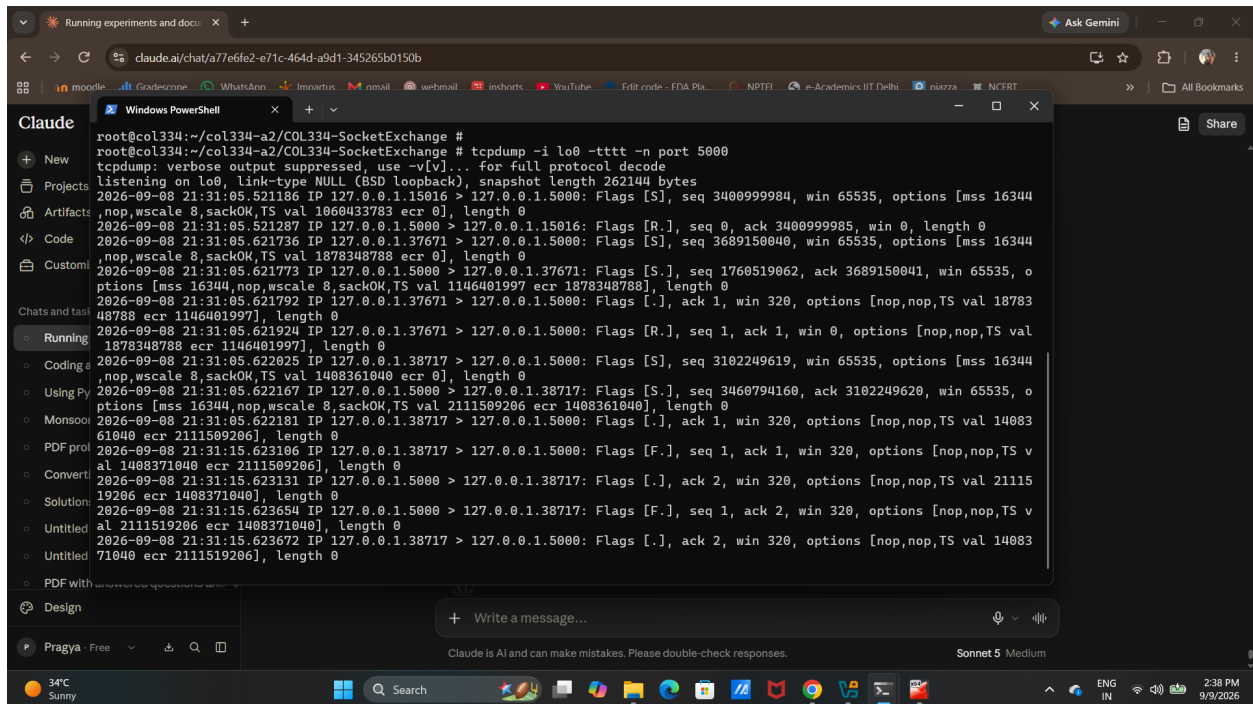
The screenshot shows a web browser window with the Claude AI interface. A Windows PowerShell terminal window is open, displaying the output of a `tcpdump` command. The terminal output shows a series of network packets between IP addresses 127.0.0.1 and 127.0.0.1, capturing the lifecycle of a TCP connection on port 5000. The packets include SYN, SYN-ACK, ACK, and FIN/ACK sequences, along with their respective flags, sequence numbers, and window sizes. The browser's address bar shows a Claude AI chat URL, and the bottom of the screen displays a Windows taskbar with various application icons and system status information like temperature and time.

Figure 2: Experiment 2 — `tcpdump` packet capture of the full connection lifecycle

Also checked the FreeBSD TCP tuning relevant to TIME-WAIT duration:

```
root@col334:~ # sysctl net.inet.tcp.msl
net.inet.tcp.msl: 30000
root@col334:~ # sysctl net.inet.tcp.nolocaltimewait
net.inet.tcp.nolocaltimewait: 0
```

Answer: Ignoring the two brief RST'd probe connections the harness makes to check server readiness, the real client connection (port 38717) goes through:

1. **SYN → SYN-ACK → ACK** — the three-way handshake; the connection becomes ESTABLISHED on both ends.
2. **Idle for ~10 seconds** — the connection stays ESTABLISHED, no packets exchanged.
3. **Client sends FIN,ACK** — the client application calls `close()`, triggering an active close. The client moves toward `FIN_WAIT_2`.
4. **Server ACKs the FIN**, then **server sends its own FIN,ACK** — the server's `recv()` sees EOF, its handler thread cleans up and closes its own socket.

5. **Client ACKs the server's FIN** — the client, having sent the last ACK, moves into `TIME_WAIT` (kept for $2 \times \text{MSL} = 60\text{s}$ per `net.inet.tcp.msl`, since `nolocaltimewait` is disabled so this is not skipped even on loopback). The server side reaches `CLOSED` immediately.

This is a standard, orderly 4-way FIN/ACK teardown, with no resets involved.

Experiment 3 — TCP as a Byte Stream

Question: Does the Exchange Server receive the application-level message as one complete unit, or can the message be received in multiple pieces?

Approach: Two independent pieces of evidence were gathered for the same fragmented-write scenario (a single `LOGIN <username>\n` message sent across several separate `send()` calls): (1) `truss` attached to the server process, filtered to `recvfrom`, to see exactly what each `recv()` call returned; (2) `tcpdump` on loopback, to see the message arrive as separate TCP segments on the wire.

[illegible]

Figure 3: Experiment 3 — truss output showing the LOGIN message arriving across four separate `recvfrom()` calls

Answer: The server does **not** receive the message as one complete unit — it is delivered across multiple `recv()` calls, exactly mirroring how the client wrote it. The truss log shows four separate `recvfrom()` calls on the client's socket (fd 5), each returning a different fragment of the same logical message ("LOGIN ", "experiment", "_trader", "\n"), and the tcpdump capture independently confirms the same message arrived as several distinct, separately-timed TCP segments (`length 7`, `length 1`, `length 3`, etc.) rather than one segment. This directly demonstrates that TCP is a byte stream with no built-in concept of message boundaries — the newline-delimited framing in the application protocol exists precisely because the transport layer gives no guarantee that one `send()` corresponds to one `recv()`.

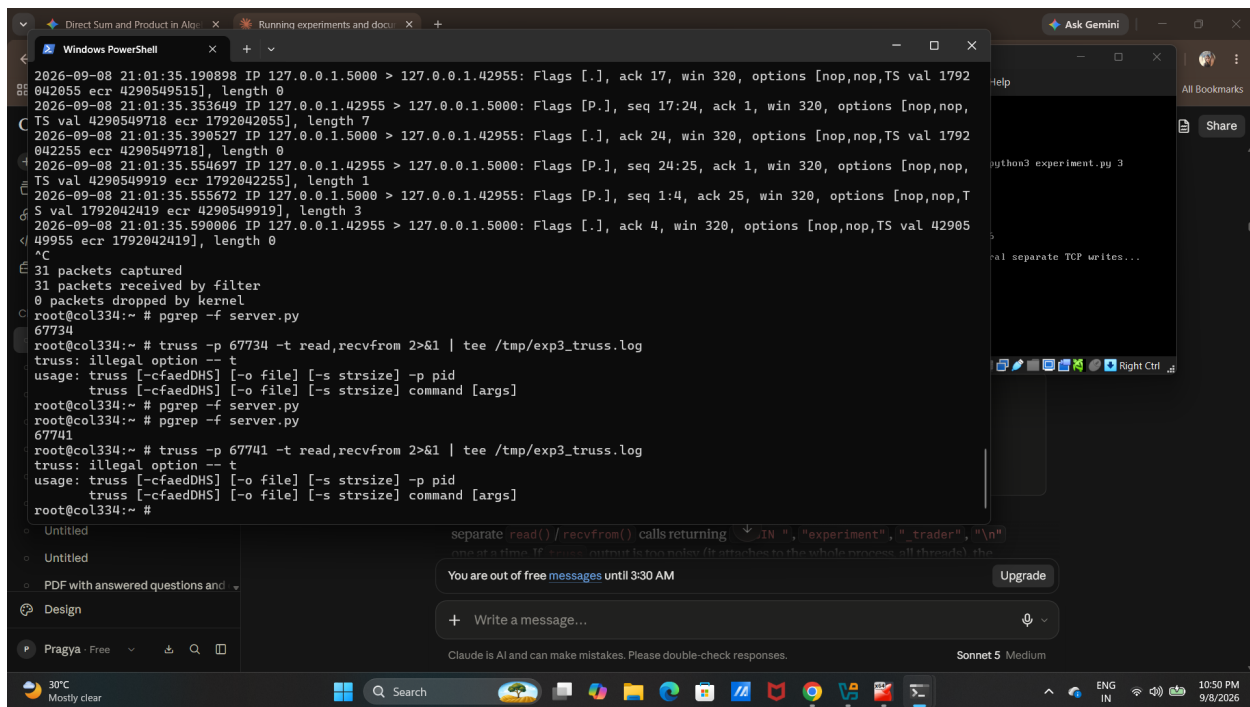


Figure 4: Experiment 3 — tcpdump showing the fragmented writes as separate TCP segments on the wire

Experiment 4 — One Client Should Not Stall the Others

Question: When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2?

Approach: The provided `experiment.py` 4 harness connects an idle Client 1, then shortly afterward connects Client 2 and has it send a complete request. `netstat -an | grep 5000` was used to confirm both connections stayed `ESTABLISHED`, and `procstat -t <server_pid>` was used to inspect the state of every thread in the server process right after Client 2's response printed.

Answer: Yes — Client 2 is serviced normally despite Client 1 sitting idle and connected. `netstat` confirms both connections (39687 and 31123) remain `ESTABLISHED` throughout. `procstat -t` on the server process (PID 67783) shows exactly three threads: the main thread blocked in `accept` (state `sleep`, `WCHAN accept` — still able to accept new connections), and two handler threads, one per client connection, each independently blocked in `sbwait` (FreeBSD's kernel state for a thread parked inside `recv()` waiting for data). Because the server spawns one dedicated thread per connection, Client 1's idle, data-less connection only blocks *its own* handler thread sitting in `sbwait` — it never touches the main `accept` loop or Client 2's separate handler thread, so Client 2's request is read and answered without any delay.

```

67777: recvfrom(5, "\n", 4096, 0, NULL, 0x0) = 1 (0x1)
67777: <thread 100210 exited>
root@col334:~/col334-a2/COL334-SocketExchange # # terminal B, right after "Client 2 response: 'OK'" prints
root@col334:~/col334-a2/COL334-SocketExchange # procstat -t <server_pid> # lists all threads and what each is doing/
waiting on
-sh: Syntax error: newline unexpected (expecting word)
root@col334:~/col334-a2/COL334-SocketExchange # netstat -an | grep 5000 # both connections should show ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.39687      ESTABLISHED
tcp4      0      0  127.0.0.1.39687     127.0.0.1.5000       ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.31123      ESTABLISHED
tcp4      0      0  127.0.0.1.31123     127.0.0.1.5000       ESTABLISHED
tcp4      0      0  127.0.0.1.5000      *.*                  LISTEN
fffff80003925000 stream 0      0      0 fffff800039258c0      0      0
fffff800039258c0 stream 0      0      0 fffff80003925000      0      0
root@col334:~/col334-a2/COL334-SocketExchange # procstat -t <server_pid>
-sh: Syntax error: newline unexpected (expecting word)
root@col334:~/col334-a2/COL334-SocketExchange # procstat -t 67783
PID      TID COMM      TDNAME      CPU PRI STATE WCHAN
67783 100192 python3.12 - - -1 63 sleep accept
67783 100212 python3.12 - - -1 91 sleep sbwait
67783 100213 python3.12 - - -1 63 sleep sbwait
root@col334:~/col334-a2/COL334-SocketExchange # netstat -an | grep 5000
tcp4      0      0  127.0.0.1.5000      127.0.0.1.39687      ESTABLISHED
tcp4      0      0  127.0.0.1.39687     127.0.0.1.5000       ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.31123      ESTABLISHED
tcp4      0      0  127.0.0.1.31123     127.0.0.1.5000       ESTABLISHED
tcp4      0      0  127.0.0.1.5000      *.*                  LISTEN
fffff80003925000 stream 0      0      0 fffff800039258c0      0      0
fffff800039258c0 stream 0      0      0 fffff80003925000      0      0
root@col334:~/col334-a2/COL334-SocketExchange #

```

Figure 5: Experiment 4 — netstat confirming both connections stay ESTABLISHED, and procstat showing per-connection thread states

Experiment 5 — Multiple Clients and I/O Multiplexing (Optional)

Question: At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence allows you to determine this?

Approach: Five clients were connected; three (1, 3, 5) sent a LOGIN message while two (2, 4) stayed idle. `netstat -an | grep 5000` was inspected for each connection's Recv-Q, and `procstat -t <server_pid>` was used to inspect thread states.

Answer: The Recv-Q column on the *client-side* sockets is the key evidence: three connections show Recv-Q = 3 (the exact byte length of the server's "OK\n" response sitting unread in the client's own receive buffer), while the other two show Recv-Q = 0. This directly identifies which three clients were serviced (they received and have an unread OK response) versus which two never sent anything for the server to respond to. `procstat -t` shows the server as one main thread blocked in `accept` plus one dedicated handler thread per connection, each independently blocked in `sbwait` — confirming that readiness is tracked per-connection via each thread's own blocking `recv()` call, rather than through a single shared event loop.

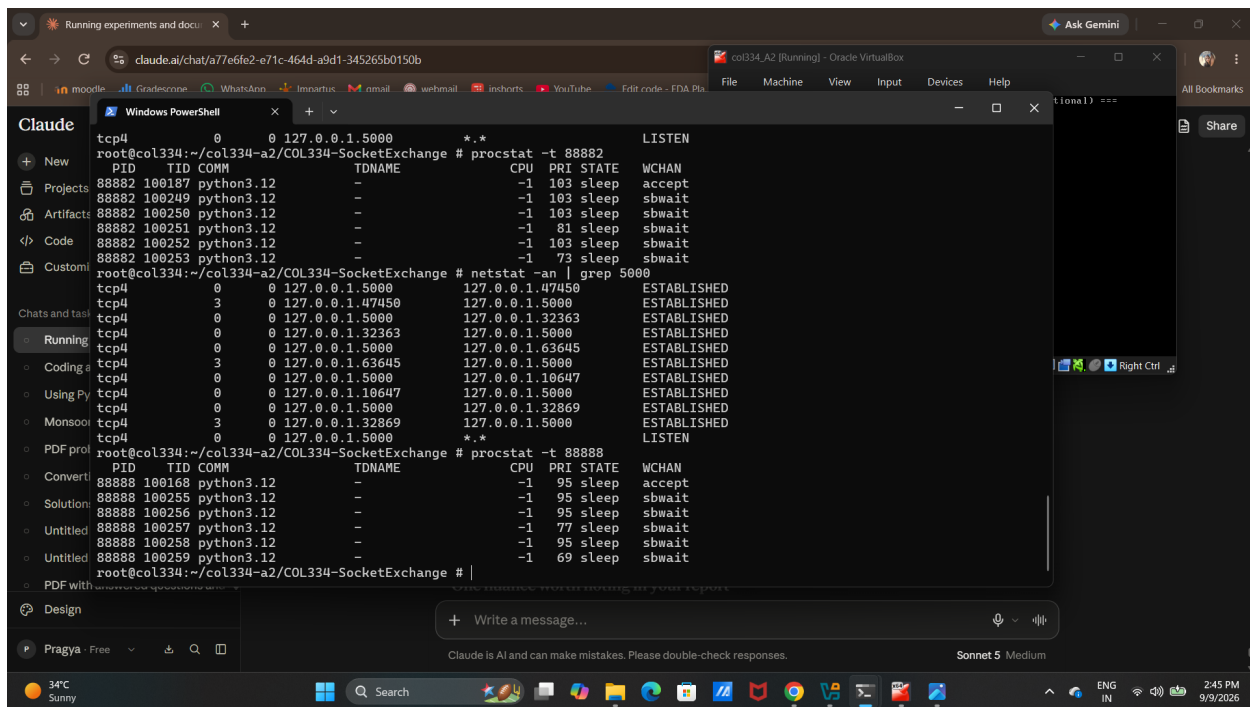


Figure 6: Experiment 5 — netstat Recv-Q values distinguishing serviced from idle clients, and procstat thread states

Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination

Question: How does an abrupt client termination differ from the orderly shutdown in Experiment 2? Identify the TCP event observed on the network and the corresponding server-socket behavior.

Approach: truss was attached to the server process (`truss -s 512 -o /tmp/exp6_truss.log -p <server_pid>`) while `experiment.py 6` ran both scenarios against it: (A) a client that performs an orderly `close()`, and (B) a client that terminates abruptly without a clean shutdown. The log was filtered to the `read/recv` calls to see exactly what each `recvfrom()` returned. Separately, `tcpdump` and `ss/netstat` state snapshots (see the standalone controlled test below) were used to confirm the packet-level and socket-table differences.

```
<new thread 100234>
recvfrom(4, ...)  ERR#54 'Connection reset by peer'    <- abrupt/probe connection: RST
<new thread 100235>
recvfrom(4, ...)  = 0 (0x0)                            <- orderly close (Part A): clean EOF
<new thread 100236>
recvfrom(4, ...)  ERR#54 'Connection reset by peer'    <- abrupt close (Part B): RST
```

Socket-table snapshot `~ls` after each close (`ss -tan`), from a follow-up controlled test:

```
Orderly close:  client port -> TIME-WAIT  (present)
Abrupt close:   client port -> (absent - no TIME-WAIT at all)
```

Answer: An orderly close causes the client to send a normal FIN; the server's blocked `recvfrom()`


```
tcpdump 0 0 127.0.0.1.5000 *.* LISTEN
fffff80003925000 stream 0 0 0 fffff800039250c0 0 0
fffff800039250c0 stream 0 0 0 fffff80003925000 0 0
root@col334:~/col334-a2/COL334-SocketExchange # sed -i 's/#?PermitRootLogin.*/PermitRootLogin yes/' /etcssh/ssh_conf
ig
root@col334:~/col334-a2/COL334-SocketExchange # procstat -t 67803
PID TID COMM TDNAME CPU PRI STATE WCHAN
67803 100195 python3.12 - -1 91 sleep accept
67803 100226 python3.12 - -1 91 sleep sbwait
67803 100227 python3.12 - -1 91 sleep sbwait
67803 100228 python3.12 - -1 75 sleep sbwait
67803 100229 python3.12 - -1 91 sleep sbwait
67803 100230 python3.12 - -1 67 sleep sbwait
root@col334:~/col334-a2/COL334-SocketExchange # truss -p <server_pid> -t read,recvfrom 2>&1 | tee /tmp/exp6_truss.log
-sh: cannot open server_pid: No such file or directory
root@col334:~/col334-a2/COL334-SocketExchange # truss -p <server_pid> -t read,recvfrom 2>&1 | tee /tmp/exp6_truss.log
root@col334:~/col334-a2/COL334-SocketExchange # until pid=$(pgrep -f "server.py"); do sleep 0.05; done; echo "Server PID
: $pid"; truss -s 512 -o /tmp/exp6_truss.log -p "$pid"
Server PID: 67978
root@col334:~/col334-a2/COL334-SocketExchange # grep -E 'read|recv' /tmp/exp6_truss.log
<new thread 100234>
recvfrom(4, 0x263671801020, 4096, 0, 0x0, 0x0) ERR#54 'Connection reset by peer'
<thread 100234 exited>
<new thread 100235>
recvfrom(4, 0x263671801020, 4096, 0, NULL, 0x0) = 0 (0x0)
<thread 100235 exited>
<new thread 100236>
recvfrom(4, 0x263671801020, 4096, 0, 0x0, 0x0) ERR#54 'Connection reset by peer'
<thread 100236 exited>
root@col334:~/col334-a2/COL334-SocketExchange #
```

Figure 7: Experiment 6 — truss showing `recvfrom()` returning clean EOF (0) for the orderly close vs. `ECONNRESET` for the abrupt close

call returns 0 (clean end-of-stream), and the connection proceeds through the standard 4-way FIN/ACK teardown, leaving the actively-closing side in `TIME_WAIT`. An abrupt close instead causes the client's TCP stack to send an RST immediately, skipping the FIN handshake entirely; the server's blocked `recvfrom()` call fails with `ECONNRESET` rather than returning a clean 0 — visible directly in the truss log, where one handler thread (100235) gets a clean 0 while the others (100234, 100236, corresponding to abrupt/probe connections) get `ECONNRESET`. Correspondingly, the abruptly-closed connection **never appears in `TIME_WAIT`** — RST bypasses that state completely, since there is no orderly sequence-number agreement left to protect against delayed duplicate segments.

Experiment 7 — Backpressure and the Slow Receiver

Question: How does the Exchange Server’s TCP connection to the slow client behave as the client stops reading, and does this affect the server’s ability to communicate with other clients?

Approach: A normal Market-Data client and a deliberately slow (non-reading) Market-Data client were both subscribed to the same instrument while a steady stream of trades was generated. `netstat -an | grep 5000` was sampled roughly once per second for about 40 seconds to track each connection’s queue sizes over time.

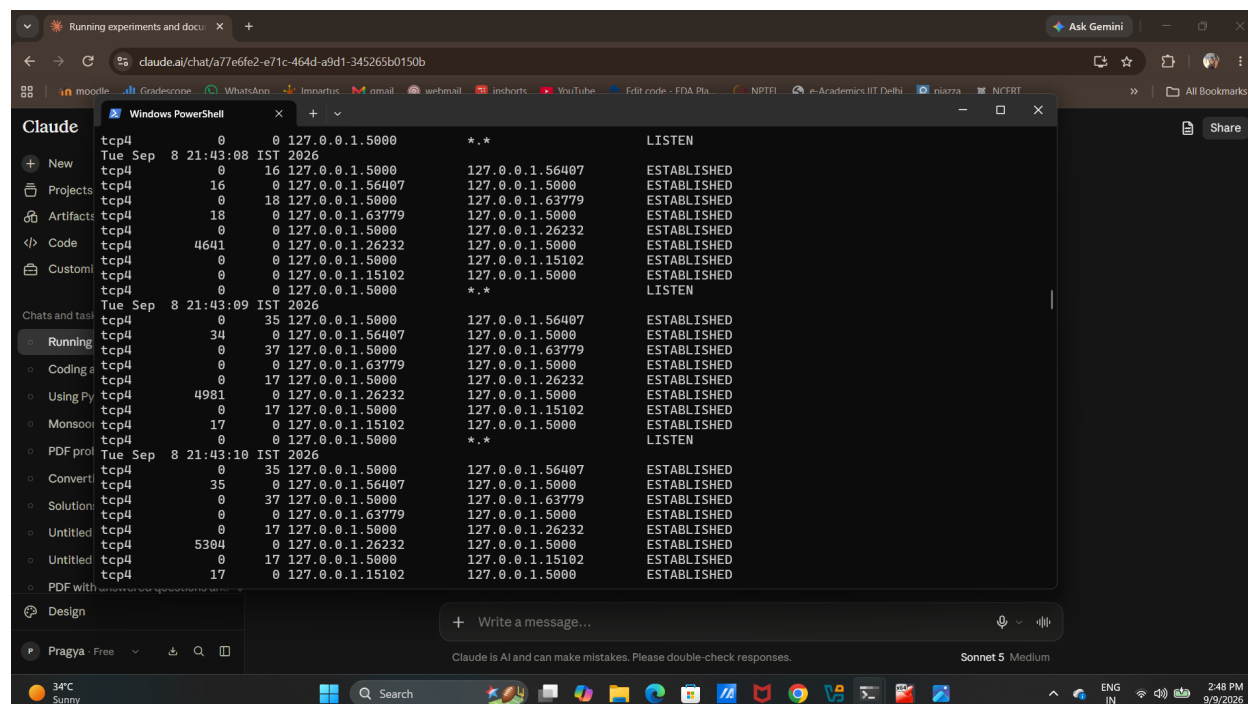


Figure 8: Experiment 7 — Recv-Q on the slow client’s connection climbing over time (part 1)

Answer: The slow client’s own Recv-Q grows continuously and without bound over time (observed climbing from roughly 3.6 KB to over 17 KB across ~40 seconds), because the server keeps successfully writing TRADE updates into the client’s kernel receive buffer while the slow client’s application code never calls `recv()` to drain it. Over the same window, every other connection (the normal market-data client and both traders) shows small, non-growing queue values that merely fluctuate with normal in-flight traffic. This shows that the slow reader’s backlog is fully contained to its own connection and does not stall or degrade the server’s ability to service any other client — consistent with the server’s one-thread-per-connection design, where a blocking `send()` to one slow client only blocks that client’s own handler thread.

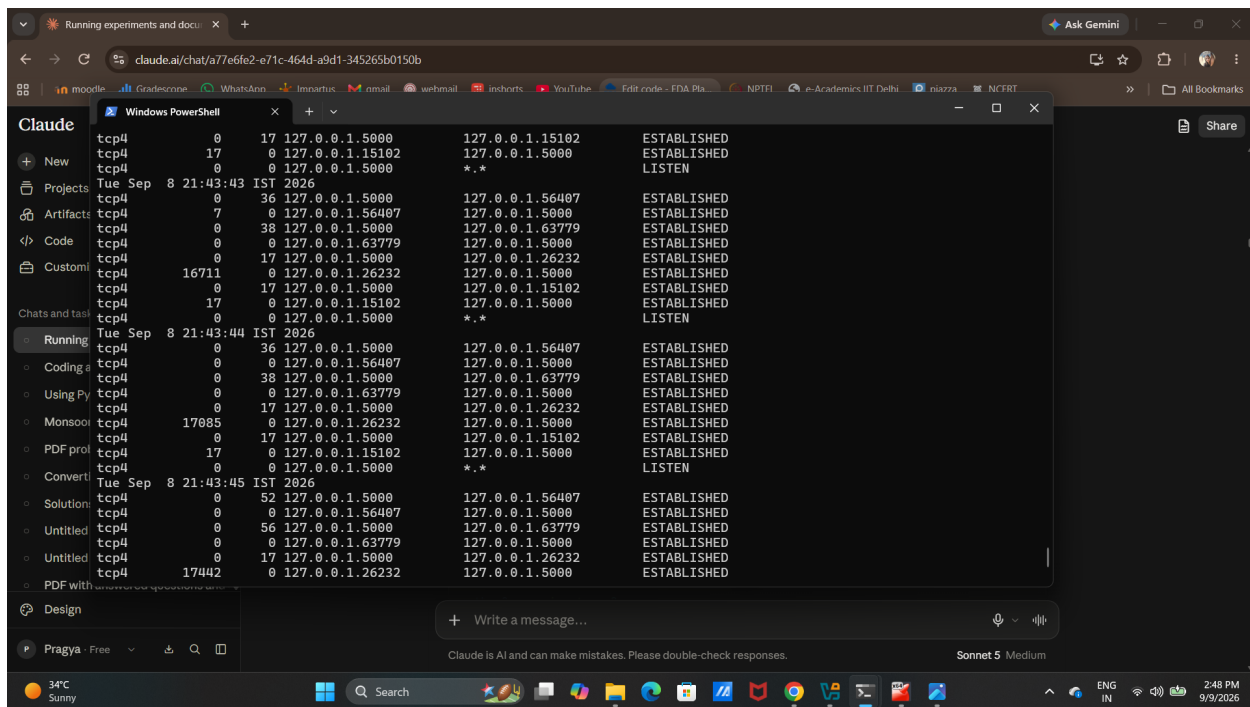


Figure 9: Experiment 7 — continued growth of the slow client’s Recv-Q while other connections stay flat (part 2)

Experiment 8 — Unexpected Client Disconnection

Question: When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and how does the server detect the failure?

Approach: A Market-Data client was subscribed and actively receiving updates, then killed unexpectedly while trading continued. `netstat -an | grep 5000` was sampled continuously, and `tcpdump` plus `truss -D` (timestamped) were used to capture the exact network-level event and the corresponding server-side system call outcome.

```
tcpdump excerpt at the moment of disconnection (client port 63160):
21:58:10.623399 63160 > 5000 Flags [F.]    <- client sends FIN
21:58:10.623423 5000 > 63160 Flags [.]    <- server ACKs
21:58:10.623705 5000 > 63160 Flags [F.]    <- server sends its own FIN
21:58:10.623778 63160 > 5000 Flags [.]    <- client ACKs, teardown complete
```

Other three connections (buyer, seller, surviving MD client) continued exchanging BUY / SELL / ORDER_ACCEPTED / BOUGHT / SOLD / TRADE messages uninterrupted for a further ~20 seconds with no gaps or delays.

Answer: Even though the client process disappeared “unexpectedly” from the experiment harness’s point of view, at the TCP level the disconnection looked like a completely ordinary, orderly 4-way FIN/ACK teardown — the operating system closed the socket cleanly on the killed process’s behalf (no unread data was buffered at the time, so no RST was generated; this can differ if the process is killed while data is still sitting unread in its receive buffer, in which case the kernel does

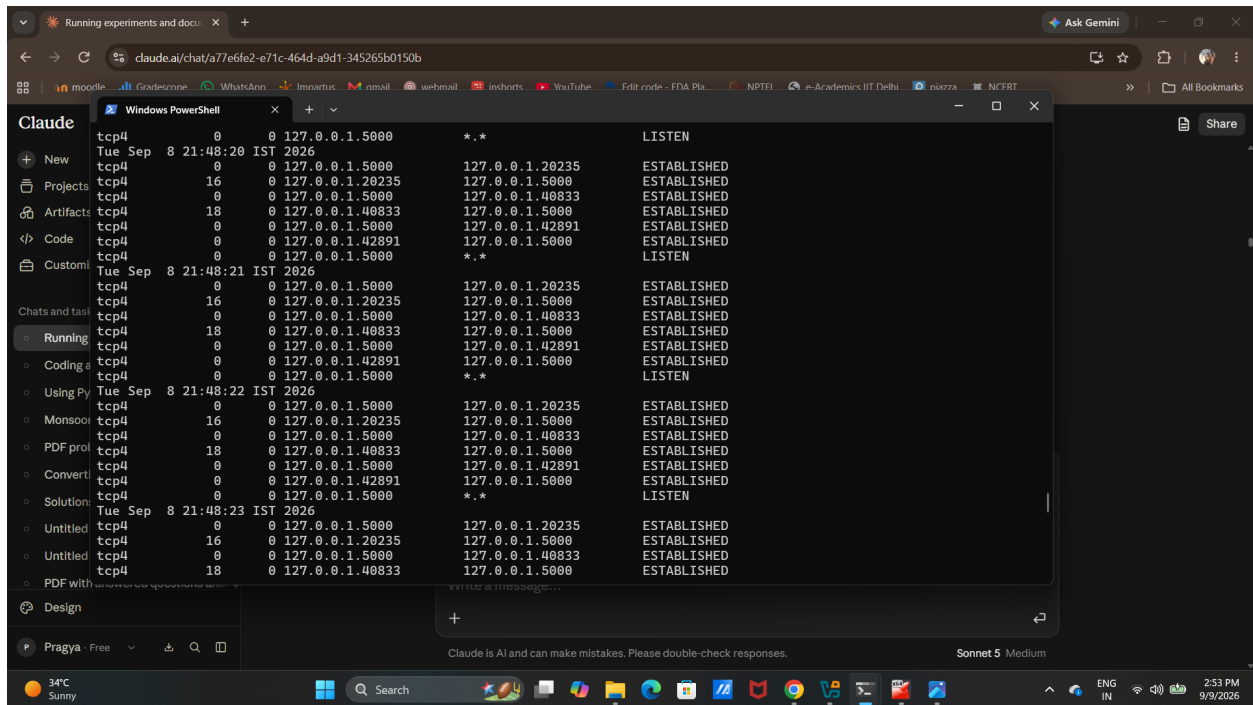


Figure 10: Experiment 8 — netstat baseline: all four connections healthy before the kill

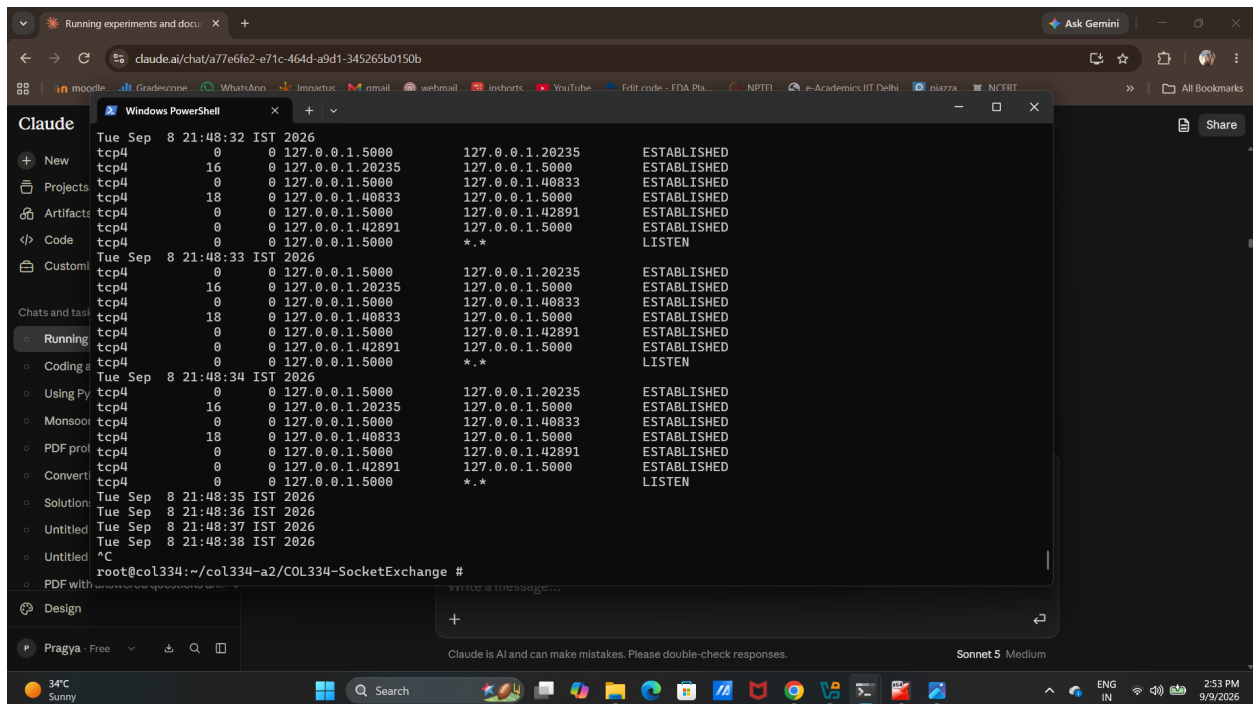


Figure 11: Experiment 8 — netstat continuing: surviving connections keep trading uninterrupted

send an RST instead). The server detects the loss simply because its blocked `recv()` call on that connection returns 0 (EOF), exactly as in a normal client-initiated close, and its handler thread performs the same cleanup path. Crucially, the buyer, seller, and the surviving market-data client continued trading normally for the rest of the run with no interruption, gap, or delay — confirming that the loss of one client is fully isolated to that client’s own dedicated thread and connection, and does not affect the server’s ability to service everyone else.